DhakaSim: the papers, the timeline, and what this build adds

Section-by-section summaries of the six published DhakaSim papers (2014 to 2025) and the preprint that preceded the last of them, a chronology of the simulator from its first demonstration to the Python build in this repository, and a statement of what that build adds and where it stands apart. ASM Nuruzzaman, 12 October 2026.

Sources, and how far each summary reaches

I read two of the papers in full: the 2018 ICT4S paper, which EasyChair publishes open access, and the 2020 preprint of the RoadBird study, which the authors submitted to the Transportation Research Board meeting and posted on arXiv two months later; the 2024 IEEE Transactions paper is its journal version. For the 2020 SIMPAT paper, the 2024 journal paper and the 2025 side friction paper I had the publishers' abstracts and the repository's own survey (`docs/research-landscape.md`), which was written from the PDFs. For the 2014 demonstration paper and the 2025 signal scheduling paper I had only that survey and, for the scheduler, the README of this repository, which documents the method as ported. Where a section below rests on an abstract rather than the full text, it says so. Every accuracy and impact figure is the papers' own.

Year	Venue	Paper	Read as
2014	Conference demonstration	DhakaSim: A Tool for Simulating Traffic Mobility in Dhaka City (Dev et al.)	survey only
2018	ICT4S, EPiC Series in Computing 52, pp. 37 to 48	Towards Simulating Non-lane Based Heterogeneous Road Traffic of Less Developed Countries (Alam et al.)	full text
2020	Simulation Modelling Practice and Theory 105, 102156	Towards simulating non-lane based heterogeneous road traffic of less developed countries using authoritative polygonal GIS map (Zubaer et al.)	abstract and survey
2024	IEEE Trans. Intelligent Transportation Systems 25(8), pp. 8486 to 8498	"To Lane or Not to Lane?" Comparing On-Road Experiences in Developing and Developed Countries Using a New Simulator RoadBird (Mushfiq et al.)	full text of the 2020 preprint, journal abstract
2025	IEEE Access 13, pp. 37344 to 37360	A Tale of Side Friction Elements in Dhaka City: From Modeling Real Cases to Revealing Impacts Through Event-Based Simulation (Rumi et al.)	abstract and survey
2025	IEEE Access 13, p. 172598, 9 October 2025	Toward Devising A Multi-Objective Traffic Signal Scheduling Approach for Non-Lane-Based Heterogeneous Traffic (Rahaman et al.)	survey and the ported code

All seven come from the Next-Generation Computing group at BUET. A. B. M. Alim Al Islam is an author on every one, and Tarik Reza Toha on five of the six.

Paper 1 (2014): DhakaSim, the first demonstration

Dev, Parvez, Bhuiyan, Mosharraf, Khan, Haque, Al Mamun, Chowdhury, Hossain, Morshed and Al Islam. Three-page conference demonstration paper, eleven authors, all at BUET. Summarised from the repository survey; the PDF is not on this machine.

ABSTRACT

The paper presents a microscopic simulator for Dhaka's traffic, which no lane-based tool reproduces. It states the architecture, lists the eleven vehicle types, and shows one sample run.

INTRODUCTION

The authors start from the observation that Dhaka's vehicles do not keep to lanes: a rickshaw, a bus and a motorbike share one carriageway and slide sideways into whatever gap opens. VISSIM and its peers assign each vehicle a lane, so they cannot show that. The tool exists to give city planners a simulator built for the traffic they have.

LITERATURE REVIEW

Brief, as a three-page paper allows. The authors name the established lane-based simulators and say why each falls short for Dhaka. The 2018 paper repeats and extends this review, and I summarise it there.

METHODOLOGY

This paper fixes the architecture that every later version keeps. A link holds segments, a segment holds strips, and a vehicle occupies as many strips as its width needs. The strip is 0.2 m wide against a lane of 2.7 to 4.6 m, so lateral movement happens strip by strip rather than lane by lane. Two implementations existed: a standalone Java application and a parallel C++ version. The four input files are the ones the current code still reads, under slightly different names (`network.txt`, `demand.txt`, `path.txt`, `paralib.txt`).

FINDINGS

One sample network of 8 nodes and 13 links, run at the stated demand and at one and a half times it. Average speed falls and average waiting time rises on nearly every link. There is no comparison with field data.

CONCLUSION

The authors claim the tool as a working platform for the city's traffic and point to calibration against real measurements as the next step. The 2018 paper takes that step.

Paper 2 (2018): pedestrians, hawkers, parking and the first calibration

Quazi Mishkatul Alam, Bejon Sarker, Biplob Biswas, Kazi Hasan Zubaer, Tarik Reza Toha, Novia Nurain and A. B. M. Alim Al Islam. ICT4S 2018, EPiC Series in Computing vol. 52, pp. 37 to 48. Funded by the ICT Division, Government of Bangladesh; the Dhaka GIS map came from BUET's Department of Urban and Regional Planning. Read in full.

ABSTRACT

Simulating the traffic of a developing country means simulating motorised and non-motorised vehicles on the same road without lanes, together with irregular pedestrians, illegal parking and lane violations. Most existing simulators cannot. The authors propose a microscopic simulator that takes a network, routes and

demand rates, animates the traffic and reports statistics. Calibrated against real traffic data, it reaches 99% accuracy on travel time.

INTRODUCTION

The argument runs from policy to tool. Planners test interventions in simulators before building them, so a city whose traffic no simulator reproduces cannot test anything. Dhaka's stream mixes buses, cars, CNG auto-rickshaws and trucks with rickshaws, vans and bicycles; the slow, human-powered vehicles behave differently and ignore lanes in particular. Pedestrians cross anywhere, hawkers walk the carriageway, and parked vehicles block it. The authors state three contributions: a simulator that divides a road into strips thinner than lanes so a vehicle can cover several and move along any of them; irregular pedestrians and hawkers, crossing the road and walking along it instead of the footpath; and a calibration against flow rate and speed measured in Dhaka with an ultrasonic sensor module (HC-001), giving 99% accuracy on travel time between two intersections.

LITERATURE REVIEW

The review covers four tools. AIMSUN (Ferrer and Barceló) offers microscopic and mesoscopic simulation with dynamic assignment, for lane-based homogeneous traffic only. MITSIM (Yang and Koutsopoulos) evaluates traffic management and traveller information systems; its open-source release of 2004 cannot be extended to non-lane traffic because its architecture assumes lanes. VISSIM (Fellendorf) supports heterogeneous vehicles and models parking and pedestrians, but all of it within lane discipline. MATSim and SUMO, the open-source options, support lane-based homogeneous traffic only. The gap the authors name is a simulator for unstructured traffic with its on-road behaviours: pedestrians on the road, parking on the road, mobile markets.

METHODOLOGY

Three modules: a core simulator, a surveillance module that collects per-link data during the run, and a signalling module that imposes signals from that data. The core loop loads parameters, network, demand and routes; works out per route when and how many vehicles to generate; generates pedestrians at random places on random links when pedestrian mode is on; generates vehicles of random sizes and speeds on random strips of each route's source link; moves everything; draws a frame; removes vehicles that reached their destination and pedestrians that crossed or were hit; and stops at the end time to write statistics.

The road network comes from five configuration files: junctions (id, coordinate, adjacent links), links (id, end nodes, segments, each segment with its two endpoints and width), routes (source, destination, link list, every entry repeated in reverse), demand (source, sink, vehicles per hour) and parameters (simulation speed and end time, pixels per strip and per metre, encounters per accident, strip and footpath strip width, maximum speed, pedestrian mode). A link is subdivided into segments to follow a curve, each segment is cut laterally into strips, and a footpath on each side occupies strips of its own. The paper states one unit of distance as a metre and one simulation step as a minute. Pedestrians are generic objects, so a static one at the kerb stands for a hawker or a market crowd, and a static vehicle at the kerb stands for an illegally parked one. Vehicles follow a car-following model and move along any strip. The implementation is Java with a Swing GUI, plus a faster console mode. A log file records every pedestrian's position and accident state and every vehicle's four corner coordinates per step, for replay. One sensor per segment, at a random point, counts and times the vehicles that pass it, and the surveillance module plots per-link vehicle count, speed and accident count with JMathPlot.

FINDINGS

Two experiments on a four-core Intel i5-3470 with 4 GB of memory. The virtual test takes a 22-link map and removes one link (link 10) at 100 vehicles per hour, a 10 km/h limit, pedestrians on and an accident probability of 0.01%: vehicle counts rise on links 9, 14 and 16 and their speeds fall, which is the behaviour one expects of rerouted demand. The real test takes the GIS map of the Sankar to Palashi route in Dhaka, runs it at 124 vehicles per hour and a 17.3 km/h limit with pedestrians on, repeats the run ten times, and compares the mean travel time in each direction with the measured one. The simulated and measured means agree to 99%.

CONCLUSION

The authors call the model deliberately simple and list what it leaves out: driver behaviour, vehicle characteristics and modal split. They had travel-time data for one pair of points only and plan to collect more pairs. They also plan to add autonomous vehicles to study their behaviour in unstructured traffic. The one-route validation is the point to hold on to: later papers measure more routes and report lower accuracy.

Paper 3 (2020): road width from a polygonal GIS map

Kazi Hasan Zubaer, Quazi Mishkatul Alam, Tarik Reza Toha, Saiful Islam Salim and A. B. M. Alim Al Islam. Simulation Modelling Practice and Theory vol. 105, December 2020, article 102156; online 30 July 2020. Abstract and survey.

ABSTRACT

The paper presents the simulator again and adds a parser that builds a simulator-ready network from a polygonal GIS map, recovering both length and width of each road. The parser averages 86% accuracy over four areas of Dhaka, and the calibrated simulator reaches 99% on travel time.

INTRODUCTION

For a strip-based simulator, width is the single most important input: it sets the number of strips and therefore the capacity. Polyline GIS data, the kind OpenStreetMap distributes, records where a road runs but not how wide it is. Authoritative city maps store roads as closed polygons, and the width is in the geometry. The paper's contribution is extracting it.

LITERATURE REVIEW

The review repeats the 2018 case against lane-based simulators and adds the map-conversion literature: tools that derive networks from GIS data do so from centrelines and either ignore width or take it from attribute tags. The survey records no other polygon-to-network converter in the review.

METHODOLOGY

The parser reads road polygons, derives a centreline and a width for each, and writes the simulator's node and link files. The authors give the algorithm, analyse its time complexity, check the theoretical runtime against measured runtime, and tune its parameters. They test it on the authoritative GIS street maps of four areas of Dhaka and then calibrate the simulator on the result as in 2018.

FINDINGS

86% accuracy on average for the parsed networks, which means about one road in seven comes out wrong. 99% accuracy on travel time after calibration.

CONCLUSION

The authors present the parser as the way to feed the simulator real cities without hand-drawing them. The 86% figure is the honest state of automatic conversion, and as far as the published record shows the parser was never applied beyond those four areas.

Paper 4 (2024): to lane or not to lane, with RoadBird

Md. Masum Mushfiq, Tarik Reza Toha, Saiful Islam Salim, Aaiyeesha Mostak, Masfiqur Rahaman, Najla Abdulrahman Al-Nabhan (King Saud University), Arif Mohamin Sadri (Florida International University) and A. B. M. Alim Al Islam. Preprint submitted to the Transportation Research Board meeting on 1 August 2020 and posted to arXiv (2010.08590) on 15 October 2020; journal version in IEEE Transactions on Intelligent Transportation Systems vol. 25 no. 8, August 2024, pp. 8486 to 8498, submitted November 2022. The preprint read in full; the journal abstract checked against it. Where the two differ, the text says which is which.

ABSTRACT

Developed countries' traffic systems are studied and run with rigour; developing countries' are not, and strategies from one may not transfer to the other. Nobody has asked how lane-based flow would perform in a city that runs without lanes. The authors use their new simulator RoadBird to test both lane-based

and non-lane-based traffic in both settings, on real topologies extracted from the GIS maps of Dhaka, Miami and Riyadh, with homogeneous and heterogeneous vehicle mixes, with and without pedestrians, and under several car-following and lane-changing models. Two conclusions: lane-based operation will not necessarily do better where traffic is now non-lane-based, and non-lane-based strategies may help a lane-based city carrying heavy mixed traffic.

INTRODUCTION

Congestion in Dhaka wastes about 3.2 million working hours a day, citing the World Bank. A simulator lets a planner test a policy before building it, but only if it reproduces the stream, and Dhaka's stream is unstructured and heterogeneous. The authors pose the question directly: what happens if Dhaka converts to lanes, and what happens if a developed city adopts non-lane operation? They extend DhakaSim into RoadBird so that one simulator can run both, and claim to be the first to make the comparison. Four contributions: the simulator itself, handling lanes and strips, homogeneous and heterogeneous mixes, pedestrians and slow vehicles; the comparison across Dhaka, Miami and Riyadh at three generation rates and varied mixes; the finding that non-lane traffic uses narrow roads with heterogeneous traffic better while lanes win on wide roads with homogeneous traffic; and the finding that non-lane strategies can help lane-based cities under heavy mixed traffic.

LITERATURE REVIEW

The longest review in the series. VISSIM, MITSIM and SUMO focus on lane-based homogeneous traffic; VISSIM's heterogeneous support still keeps lanes, and Huang and Aghabayk show how it is used for conflict analysis and calibrated by multithreading. For Indian traffic, Vedagiri's HETEROSIM estimates saturation flow in passenger car units per metre of width and finds it linear in width, and Arasan proposes area occupancy in place of density because density assumes lanes. Arasan and Koshy give a non-lane simulation framework; Jin adds lateral separation to a full velocity difference car-following model; Muniruzzaman calibrates a non-lane microsimulation for Dhaka but does not compare it with lanes. Ngoduy, Yang, Nokandeh and Li study heterogeneous flow among motorised vehicles only, with cars and trucks, and Nokandeh's dynamic PCU keeps lane discipline. Mathew's SiMTraM divides a lane into strips and can run both regimes but has no random pedestrians or non-motorised vehicles; Agarwal's agent-based queue model, Mohan's second-order continuum model calibrated in Chennai and Chand's saturation flow model at Indian signals all leave those out too. The gap the authors name is a comparison of lane and non-lane operation under a stream that mixes motorised and non-motorised vehicles.

METHODOLOGY

The preprint implements three car-following models: a Newtonian model with hard braking, where a vehicle accelerates at a constant rate or brakes to the gap ahead in one time step of 1 s; Gipps' model, with no explicit braking, so vehicles can collide; and a hybrid that drives on Gipps' speed and applies the hard brake when a collision is imminent. The journal version widens this to thirteen car-following and four lane-changing models, which is the set the Python port carries. Three discretionary lane-changing models: a straightforward one that shifts when the target strip has room, Gipps' with feasibility from the braking the subject and the target follower would need and gap acceptance by an exponential probability of the lead and lag gaps, and a GHR-based one that wants to change when the GHR acceleration goes negative (sensitivity 15, speed exponent 1, headway exponent 2). Vehicles arrive with negative-exponential headways. A link has floor(width / strip width) strips and a vehicle takes ceil(vehicle width / strip width) of them, so a 0.5 m strip gives non-lane operation and a 2.5 m strip, wider than any vehicle, gives lanes. Four metrics: average speed on a link, average waiting time on a link (time stopped), average flow across the link's midpoint, and average vehicle speed over the trip.

Topologies for Dhaka, Miami and Riyadh were extracted from GIS maps. Generation rates were 100, 400 and 800 vehicles per hour on Dhaka and 500, 1000 and 2000 on Miami and Riyadh. The Dhaka mix is 55% slow (bicycle 9%, rickshaw 89%, van 2% of that share), 40% medium (CNG 83%, bus 15%, truck 2%) and 5% fast (motorbike 88%, car 12%); the Miami and Riyadh mix is 9%, 75% and 16%. Each scenario ran 1800 s on seeds 1 to 10 and the graphs show the mean. Miami is included because its traffic becomes lane-less during a hurricane evacuation.

VALIDATION

The route is Shankar to Palashi in Dhaka, 4.3 km, 8 intersections, 18 links, both directions, cars and motorbikes. The day is split into low (22:00 to 09:00), medium (09:00 to 16:00) and high (16:00 to 22:00) density. The authors logged 63 live travel times from Google Maps, 34 before the pandemic and the rest during it, using pandemic data for low and medium density only, and ran 45 simulations of 40 minutes, 15 per density. A two-sample t-test and a Kolmogorov-Smirnov test at 5% give p above the threshold in 9 of 12 cases. Mean error shows no systematic bias. Mean absolute error runs from under a minute to 4.6 minutes, root mean square error up to 5.1. Accuracy by MAPE runs from 82% to 95%, mean about 88%; by RMSPE the mean is about 85% with a best of about 94% and two cases near 75%. The three failures are motorbikes at low density towards Palashi and both motorbike cases at medium and high density towards Shankar.

FINDINGS

- On Dhaka's topology, link speed falls as the generation rate rises and stays below 10 km/h, which matches the World Bank's figure of about 7 km/h for the city. Non-lane speed exceeds lane speed at every rate: the roads are narrow, the vehicles small, and the slow vehicles set the pace.
- Waiting time rises with the generation rate and is lower without lanes. Switching the mix to homogeneous, which removes the slow vehicles, improves speed, waiting time and flow for both regimes.
- Pedestrians change the heterogeneous results little at medium rate, and slow the homogeneous ones slightly.
- On average vehicle speed, non-lane wins at the low rate but lanes win at the high rate: the one crossover in the study, which the authors say matches their experience of structured traffic under load.
- On Miami's topology, with the roads of the chosen area also narrow, Dhaka's non-lane mix moves more vehicles with less waiting than Miami's own mix, and the gap is widest in the non-lane network.
- On Riyadh's wider, longer roads, lane-based traffic wins on both flow and waiting time.

CONCLUSION

Converting Dhaka to lanes does not guarantee improvement, because its vehicles are small, slow and human-powered and its roads narrow; wide networks do benefit from lanes. Non-lane strategies may suit emergency evacuations, where contraflow and lane reversal are the usual tools and the stream mixes fast and slow vehicles. Future work named by the authors: richer parameters covering safety, intersections, signalling and pedestrians; a combined setup with lane and non-lane links in one topology; and evacuation scenarios where demand exceeds capacity.

Paper 5 (2025): side friction, from field cases to simulated impact

Ahmed Mahir Sultan Rumi, Hasan Nahiyan Nobel, A. H. M. Osama Haque, Abu Humayed Azim Fahmid, Tarik Reza Toha, Nouf Alaloula, Najla Al-Nabhan, Abdulaziz Alnamlah and A. B. M. Alim Al Islam. IEEE Access vol. 13, 2025, pp. 37344 to 37360, doi

10.1109/ACCESS.2025.3544511. Abstract and survey.

ABSTRACT

Side friction elements shrink the effective width of Dhaka's streets and block them. No study has modelled those elements and measured their impact in a simulator built for heterogeneous non-lane traffic. From real data the authors list the elements with substantial impact (roadside objects, pedestrians and non-motorised vehicles), model each with parameters that capture its behaviour, and extend DhakaSim with them. They validate the extended simulator against Google Maps data and run an impact study. Side friction can cut Dhaka's average speed by up to 36.55% and raise average waiting time by up to 161.3%. Pedestrians do far more damage than any other element and rank among the leading causes of congestion. Non-motorised traffic lowers average speed but keeps flow up on low-capacity roads.

INTRODUCTION

The authors frame side friction as the gap between a road's nominal width and the width traffic can use. Previous side-friction studies come from lane-based settings and from Indian arterials; none models Dhaka's elements in a simulator that reproduces its traffic. The paper claims the updated DhakaSim as the only simulator that captures the city's parked vehicles and irregular pedestrians together.

LITERATURE REVIEW

From the abstract and survey: the review covers field studies of side friction on urban arterials in South Asia, which measure speed loss against roadside activity, and the simulation literature, where VISSIM-style tools model parking and pedestrians within lane discipline. The gap is a contextually appropriate simulator with the elements modelled from field data.

METHODOLOGY

Seven element types in three groups: parked cars, parked rickshaws and parked CNGs; standing pedestrians, pedestrians walking along the road and pedestrians crossing it; and non-motorised vehicles. Field observation on Dhaka streets fixes each element's parameters (density, dwell, movement). The simulation is event-based: objects appear, block strips for a time and clear; pedestrians arrive at their own rates per class. Validation is against Google Maps travel times, as in 2024. The impact study switches elements on and off and measures speed, waiting time and flow.

FINDINGS

Up to 36.55% lower average speed and up to 161.3% longer average waiting time with all elements present. Crossing pedestrians are the single largest source of congestion. Removing non-motorised vehicles raises speed and lowers throughput at the same time, because rickshaws pack the limited road space efficiently. Note the shift in tone from 2024, where pedestrians barely moved the heterogeneous results: the two papers model pedestrians at different detail and density, and anyone building on them should check which pedestrian parameters a run carries.

CONCLUSION

Side friction is a first-order effect on Dhaka's speed, and the pedestrian element is the one to manage first. The rickshaw finding cuts both ways and the authors present it as such.

Paper 6 (2025): multi-objective signal scheduling for non-lane traffic

Masfiqur Rahaman, Ahmed Mahir Sultan Rumi, M. S. Islam, Tarik Reza Toha, Md. Masum Mushfiq, M. S. Rahman, M. A. Nayeem, Najla Al-Nabhan and A. B. M. Alim Al Islam. IEEE Access vol. 13, 2025, from p. 172598, published 9 October 2025. A preprint, "A Multi-Objective Approach for Traffic Signal Scheduling Leveraging Vehicle Priority Model and Microscopic

Simulation" (also circulated as "... Using NSGA-II For Developing Cities"), appeared in April 2025 with a partly different author list. Survey and the ported code.

ABSTRACT

Nearly all signal scheduling research assumes lane-based homogeneous traffic. The authors ask what an optimal schedule looks like when neither holds, compare binary-encoded and real-encoded NSGA-II on a numerical simulator, find real encoding better, propose two objective functions built around the heterogeneous stream together with a vehicle priority model, and integrate the result into DhakaSim. Against the fixed-time schedules Dhaka uses, the method raises average speed by 163.1% and cuts average waiting time by 56.1% across Dhaka's network at peak hours, and improves Miami and Riyadh as well.

INTRODUCTION

Fixed-time signals are what the few working signals in Dhaka run, and they ignore the queue. A signal that serves an approach full of rickshaws for the same green as one full of cars under-serves it, because a rickshaw takes more than twice as long to clear a stop line. The contribution is an optimiser that knows the difference.

LITERATURE REVIEW

The review, per the survey, covers genetic and multi-objective signal timing from Sun, Benekohal and Waller's NSGA-II formulation onward, reinforcement-learning controllers, and the heterogeneous-traffic signal studies from India, all of which assume lanes or ignore non-motorised classes. The gap is scheduling for a non-lane heterogeneous stream with class-aware objectives.

METHODOLOGY

A schedule is one green duration per approach. NSGA-II searches schedules against two objectives at once and returns a Pareto front; a weighted choice (weight W) picks one schedule off it. Version 1 of the objectives counts every vehicle alike: Equation 1 scores the queue that fails to clear, Equation 2 the cycle length. Version 2 prices the vehicle classes apart with a motorised weight w (Equation 3) and scores queue-weighted red time against total queue (Equation 4). Candidates are scored on a numerical simulator, not on DhakaSim: an isolated junction whose approaches discharge at 0.9 motorised and 0.4 non-motorised vehicles a second for as long as they hold green, cheap enough for tens of thousands of evaluations per decision. DhakaSim then runs the chosen schedule. Each junction re-plans when its cycle ends, against the queue at that moment. Both encodings were tried; real encoding won and was integrated. Experiments ran on the Dhaka, Miami and Riyadh networks of the 2024 paper against fixed-time and biased-random baselines.

FINDINGS

163.1% higher average speed and 56.1% lower average waiting time on Dhaka at peak against fixed time; the preprint reported 146.9% and 66.4%. Miami and Riyadh improve too, which the authors read as generalisation. The paper reports version 2 as the stronger objective set and finds that the motorised weight should be low and that the front-selection weight barely matters. The authors note that a real deployment would have to add driver reaction time to every allocated green.

CONCLUSION

A class-aware multi-objective scheduler beats fixed time on a non-lane heterogeneous network by a wide margin. The paper schedules offline per cycle; adaptive control that responds to live detection is the follow-up, and the group has since posted work on camera-based detection at Palashi.

How DhakaSim evolved

Three rebuilds in eleven years, each by a new group in the same lab, each keeping the strip model and adding one layer. Then, in 2026, a port to Python that kept the arithmetic and rebuilt everything around it.

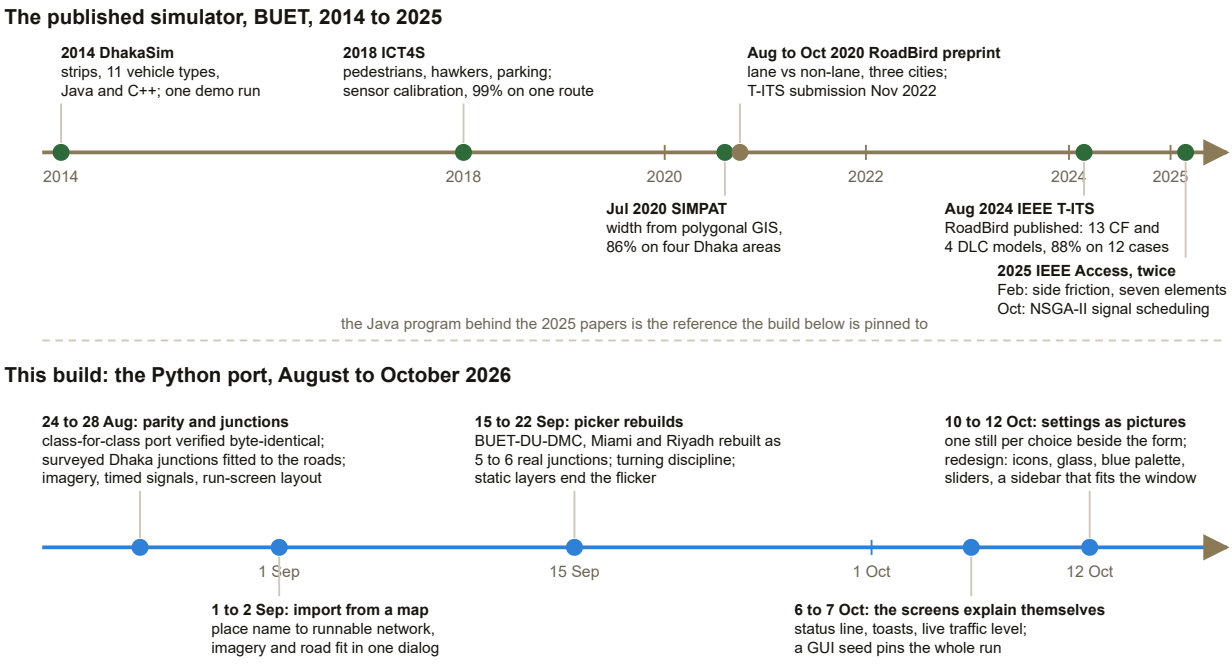


Figure 1. DhakaSim from the 2014 demonstration to the current build. Paper dates are publication dates; the 2026 dates come from this repository's commits and working notes.

When	Phase	What changed
2014	DhakaSim, first demonstration	Links, segments and 0.2 m strips; eleven vehicle types; Java and parallel C++; one 8-node run, no field data.
2018	Second rebuild, ICT4S	Pedestrians crossing anywhere, hawkers on the carriageway, parked vehicles; five input files; per-segment sensors; ultrasonic field data; 99% on one route.
2020	GIS parser, SIMPAT	Width recovered from polygonal maps; 86% over four areas.
2020 to 2024	RoadBird, T-ITS	Strip width as the lane switch; thirteen car-following and four lane-changing models; Dhaka, Miami, Riyadh; twelve-case statistical validation, 88% mean.
2025	Side friction and signals, IEEE Access	Seven side-friction elements from field data; NSGA-II scheduling with class-aware objectives; the Java code these two papers ran is the reference for the port.
Aug 2026	Python port, parity	Standard-library Python, class for class, byte-identical to the Java original under a matched seed; four surveyed Dhaka junctions with hourly demand and mix; imagery; links fitted to real roads.

When	Phase	What changed
Aug to Sep 2026	Junctions and places	Medians, roundabouts, stop lines read off the drawn junction, keep-clear entry, turning discipline, signal bars, 20 s greens; map import with a junction picker; three multi-junction networks rebuilt from OpenStreetMap.
Oct 2026	A research instrument with a face	Status and toast feedback, live traffic level, a seed that pins everything, one preview picture per choice, a redesigned start screen, a run-screen sidebar that fits the window.

What the current DhakaSim adds

The papers' simulator is a Java program that one lab group hands to the next. The build in this repository is a port of that program with its arithmetic kept and everything around it rebuilt. The improvements fall into six groups.

1. A port you can trust, and prove

Every module was translated class for class and checked against the Java original: with a matched seed the two produce byte-identical CSV output and printed metrics across all thirteen car-following models, all four lane-changing models, the pedestrian and roadside-object paths, the accident log and the drawing geometry. Java's arithmetic quirks (NaN-propagating min and max, half-up rounding, truncating casts, sign-of-dividend modulo) are reproduced on purpose and pinned by tests. One command, `hash_run.py`, runs a seeded 90-step run of every shipped network and compares every output against a recorded baseline, so any drift in the model shows up as a named file. The suite stands at 352 tests today. Every change that moves the model since August has been shown to be a pure addition first: the new mode switched off, hashes unchanged, then recorded. No paper in the series reports a test suite or a parity check.

2. Junctions that are geometric objects

In the Java original every arm of a junction ends at the same point, the signal cycles every simulation step, and queues form on top of the junction. That is what the owner saw as signal bars "clogged within the intersections". The build reads each junction's shape from a `geometry.txt`: solid medians, one-way arms, and roundabouts whose arms are cut back to the ring so traffic circulates instead of driving across the island. Stop lines are cast from the drawn outline of the junction patch, fillets included, so a red light holds the queue at the edge of the box, and a box deeper than a short mouth segment walks its line back up the approach. A vehicle enters on green only when its exit has room and no standing queue reaches back into the box; a vehicle caught past the line by a phase change clears the box rather than parking in it. Turns are capped at a speed that falls with the turn angle and held on one line across the box. Each of these is a switch with a Java-parity Off setting, each was measured (banani completes about 27% fewer trips under the discipline at matched 20 s greens, because green time now pays for the approach), and the 1 s box-packing of the original out-throughputs disciplined signals everywhere, which is a finding about oversaturated junctions rather than a bug. The 2024 paper lists intersections and signalling as future work; this build has them.

3. Real places, with real demand

Four surveyed Dhaka junctions ship with the build (Banani 23, Banani 27, Bijoy Sarani and the two-signal Kakrail corridor with its roundabout), each with 24-hour turning counts, an hourly demand profile and an

hourly vehicle mix, so a run can be set to the busiest hour or any other. The 2024 paper's three cities ship too, rebuilt from OpenStreetMap as five or six real junctions each through the same importer any user can run: type a place name or paste a map link, click a junction, choose which legs to keep, and get a runnable network with imagery and its links bent onto the real roads. The 2020 paper got width from polygons at 86%; this build takes it from map tags and bends the centrelines onto the roads instead, with the georeference anchored on a single control point and checked by a test that a junction node sits within metres of the nearest mapped road. The map behind the plan view is the OpenStreetMap rendering, with named streets a reader can check the fit against.

4. The 2025 scheduler, ported and re-measured

Fixed-time, biased-random and both multi-objective controllers are in the build, with the paper's numerical simulator, its real-encoded NSGA-II and its two weights. Measured on the surveyed junctions against the best of the paper's fixed-time greens rather than any single one, the first objective set wins where the traffic is most mixed and loses where it is least, and biased-random is worst everywhere, both as the paper has it. The second set, which the paper reports as stronger, is weaker here on three networks of four, and the build documents why: Equation 4 reduces to the cycle length minus a queue-weighted mean green, which for a fixed cycle is smallest when the whole cycle goes to the busiest approach, so the objective starves the quiet ones. Three readings of that equation were implemented and the best kept. The scheduler never draws from the simulation's random stream, and a test trips if it does.

5. Reproducibility and measurement

The Java original left roadside objects, pedestrian arrivals and blockage draws on clock-seeded generators, so a seeded run was only partly repeatable. Here a seed, typed in the GUI or passed on the command line, pins all of it. A parallel sweep harness runs scenario grids; the five error measures and the two hypothesis tests of the 2024 paper's validation are implemented without SciPy; a paper-comparison report tests ten of that paper's claims against a sweep; and a live traffic-level multiplier loads a junction up or eases it off while the run plays. Side-friction density scales with network length, which the original did not do and which put a kilometre of Dhaka's parking on a four-link grid.

6. Seeing the run

A 2D plan view over imagery and a 3D perspective view of the same run, switchable mid-run with a camera tween; a self-contained HTML report per run with animated 2D and 3D replays, a dashboard and the run's settings; a start screen that previews every choice of every setting as a picture beside the form (the four signal controllers side by side, the two geometry modes, the strip grid at three widths), with a status line that says what Start will run and what has changed from the place's defaults; and a run screen whose sidebar fits the window at the largest type that stands in it. The 2018 paper had a Swing window and JMathPlot graphs.

> **What this build still does not do.** It has no polygonal GIS parser, so the 2020 paper's method is not reproduced. It has no field travel-time data, so the 2024 validation table cannot be re-run. The scheduler's headline gain is not reproduced, for the reason given above, on single-junction networks over 480 s rather than the paper's whole-city network over 30 minutes. It measures speed, waiting time and flow, as every paper does, and not safety or emissions.

The build against Nielsen's ten usability heuristics

The owner set Nielsen's heuristics as the basis for judging the screens on 6 October 2026, and the start screen and run screen were rebuilt against them over the following week. The table takes each heuristic in turn, names what in the build answers it, and names the gap that remains. The judgement is mine, from the code and from driving both screens; nobody has run a user study. The papers' own simulator had a Swing window with configuration files and JMathPlot graphs, so most rows below describe things no earlier version had.

Heuristic	What the build does	Gap
1. Visibility of system status	The start screen's footer says what Start will run, counts the settings changed from the place's defaults, and marks each changed row with a dot. The run screen's toolbar shows the step, the end time and the view every step; a toast on the picture names every zoom, view change, map toggle and traffic change; the bottom bar shows progress. The import dialog has a progress bar fed by the worker's own log lines, and the Fetch map button says it is fetching.	A long preview build shows "Building…" in the cell but no estimate. The scheduler's per-cycle re-planning is invisible while a run plays.
2. Match between system and the real world	Settings are worded as a reader thinks: Playback speed in Step, Slow, Normal, Fast and Max rather than a frame delay in milliseconds; Real geometry explained as medians and a roundabout ring, not "Java parity"; places by name with a sentence on their surroundings; the signal controllers named by what they do. Speeds in km/h, greens in seconds, traffic as a multiple of the survey.	Strip width is still a modelling term; the preview picture explains it, the name does not. "Multi-obj. v1" and "v2" are the paper's labels, not a reader's.
3. User control and freedom	Reset to defaults on the start screen; Reset on the zoom; New simulation returns to the form at any point in a run, asking first if the run is unfinished; a view change can be reversed with the same key; the import picker has Close and Rebuild; an imported network can be removed, a shipped one cannot. Typed values are never rewritten under the typist.	Closed on 15 October 2026: Ctrl+Z takes back the last change, a click on a row's name takes back that row's, and the footer names what came back. Still no redo.
4. Consistency and standards	One palette and one set of controls across the start screen, the run screen and the HTML report; the same five-detent slider sets playback speed on both screens; the same row icons appear in the form and in the run-screen sidebar; every strip is a `_Segmented` row of cells and every switch an Off/On pair; Space, V and M are named on their buttons.	The zoom and traffic sliders are Tk scales while playback speed is a drawn slider; they look alike but behave differently under the keyboard.
5. Error prevention	A bad value is named in red as it is typed and Start refuses with the same sentence, so a traceback never reaches the reader. The junction picker turns its status red when a chosen junction has fewer than three legs, and refuses a set of junctions that is not connected before anything is fetched. The map fetch refuses to overwrite hand-placed imagery. A value the slider does not offer is kept, not snapped.	A seed of -1 and a trace replay of another place's recording are allowed; both are explained in the preview rather than prevented.

Heuristic	What the build does	Gap
6. Recognition rather than recall	Every choice of every setting has a picture beside it: the four signal controllers side by side, the two geometry modes, the strip grid at three widths, the map of each junction. The footer shows a row's explanation while the pointer is on it. The run-screen sidebar keeps the run's settings in view for the whole run, with the same icons as the form.	The pictures need a wide window; at 1280 px the form scrolls beside a narrower column and the pictures shrink.
7. Flexibility and efficiency of use	Keyboard shortcuts for pause, view and map; wheel zoom and drag pan; a per-network defaults file, a `--set` override and a headless command line for everyone who does not need the window; a sweep harness for experiments; the Randomise button for a fresh seed.	No saved presets of a whole form, and no recent-places list.
8. Aesthetic and minimalist design	The form chooses its density by measurement so nothing is clipped and nothing is wasted; captions move to the footer when they do not fit; the Scale row left the form because it meant nothing to a reader; the sidebar sentence under the map toggle went for the same reason; the picture column takes a third of the window and no more.	Thirteen junction tiles on a laptop push the form to a tight rung; the sections still carry two-line captions on a large monitor.
9. Help users recognise, diagnose and recover from errors	Error sentences name the setting and the rule ("End time must be a whole number of 1 or more"), in the same place the status is shown. The import dialog reports which stage failed and leaves the network runnable when only the map or the road fit failed. A trace recording of another place is explained in the slot rather than shown as an empty road.	A failed Overpass fetch reports the last mirror's error text, which is a server's sentence, not a reader's.
10. Help and documentation	Row explanations in the form, a sentence on each place's surroundings, toasts that say how to move in each view, a README that documents every parameter and quirk, and a per-run report that explains every result and term.	Closed on 15 October 2026: F1 and a Help button on both screens open a page built from the same table the footer reads. The page is text; it has no pictures of the screens yet.

Four heuristics were the ones the owner named in October, and they are the four the build answers best: status, recognition, error prevention and control. The two weakest rows were undo and in-window help; both were closed on 15 October 2026 with a per-setting history behind Ctrl+Z and a help page behind F1, and the gaps that remain in those rows are a redo and pictures on the help page.

Where it stands apart

1. **Parity as a contract.** This is the only version of the simulator whose agreement with its predecessor is a test rather than a claim. Every behavioural addition carries an Off setting that restores the Java output byte for byte.
2. **Junctions with shape.** The papers' networks converge their arms on a point and signal every step. This build is the only one where a stop line is derived from a drawn junction, a roundabout is a ring, and entry waits for room on the exit.
3. **Surveyed demand by the hour.** The papers generate demand from a rate per route. This build carries turning counts for four Dhaka junctions by hour of day, with the vehicle mix changing with the hour.

4. **Any place from a map, in one dialog.** Earlier versions needed a parser run by its authors or networks drawn by hand. A user of this build imports a junction from a place name, with imagery and a road fit, and the result is a start-screen tile.
5. **A documented failure mode in the published objective.** The build does not take the paper's stronger objective on trust; it measures it against a harder baseline, finds it weaker, and writes down the algebra that explains it.
6. **Everything seeded, no dependencies.** Standard-library Python only, so it runs anywhere Python does, and a seed reproduces a run to the byte, including the parts the original left to the clock.
7. **A settings screen that teaches the model.** Nobody has to know what a 0.5 m strip means before pressing Start: the picture beside the setting shows the grid, the signal row shows four junctions under four controllers, and the footer says what will run.

Written from the two open-access full texts (ICT4S 2018; the RoadBird preprint, arXiv 2010.08590), the publishers' abstracts for the SIMPAT, T-ITS and side-friction papers, the repository's survey of all seven PDFs, and this repository's README, CLAUDE.md, commits and session notes. Figures attributed to a paper are that paper's own. DhakaSim Python build, 12 October 2026.